



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Persistencia, Metadatos y Diseño de Sistemas de Archivos Plano (PicoFS)

Práctica 8

Presenta:

Rojo Peña Manuel Ianluck

Profesor:

Javier León Cotonieto

Ayudante laboratorio:

Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:

4 de mayo, 2026

Introducción

En esta práctica sdiseñaremos e implementaremos una **PicoFS**, un sistema de archivos plano alojado en los últimos 256 KB de la memoria Flash del microcontrolador RP2040. Con el objetivo de comprender de primera mano cómo un espacio lineal y físico de silicio se convierte en una estructura organizada de archivos accesible mediante funciones de sistema, y cuáles son las restricciones reales que la arquitectura del hardware impone sobre cada operación de lectura, escritura y borrado.

La premisa que articula todos los módulos es la asimetría física de la memoria Flash **NAND/NOR**: escribir requiere que los bits pasen de 1 a 0, mientras que resetearlos a 1 exige aplicar un alto voltaje sobre un bloque completo de 4096 bytes, sin importar cuántos de esos bytes se deseen cambiar realmente. Esta restricción convierte cada modificación en un ciclo costoso de *Read-Modify-Write* y obliga al diseño del sistema de archivos a minimizar la cantidad de borrados físicos mediante estrategias como el borrado lógico.

Se organizó el reporte en cuatro módulos que van desde la validación de la identidad del sistema al montarlo (**Módulo 1**), pasando por la asignación alineada de bloques (**Módulo 2**), la escritura y lectura persistente de datos (**Módulo 3**), hasta el borrado lógico y el diagnóstico visual del estado del sistema (**Módulo 4**). A estos módulos le agregamos una sección de **Conocimiento Arquitectónico** que responde preguntas de análisis más profundo, y una sección de **Diagnóstico** con el análisis de los resultados de la ejecución.

Módulos de Implementación y Análisis Arquitectónico

Módulo 1: Inicialización y Firma del Sistema (Superbloque)

Reflexión: El archivero y la marca de identidad

Para determinar si un sistema de archivo físico ha sido previamente auditado y organizado, la estrategia más eficiente consiste en reservar siempre la primera posición del primer cajón para un documento con un número identificador único. De este modo, al inspeccionar únicamente este primer elemento, es posible deducir en tiempo constante ($O(1)$) si el mueble entero posee el formato correcto o si debe ser vaciado y reestructurado. Incluso al escalar el problema a miles de archiveros, el tiempo de verificación sigue siendo lineal respecto al total, pero la inspección individual se mantiene ágil y determinista gracias a que la ubicación de la marca de identidad es predecible y estricta.

Validación de identidad en el kernel

En el diseño de sistemas operativos, este concepto se formaliza mediante el **Número Mágico** (*Magic Number*). Este identificador estático se almacena dentro de una estructura de metadatos crítica denominada *superbloque*, la cual reside en un desplazamiento (*offset*) constante y predefinido dentro de la partición. Cuando el núcleo (*kernel*) intenta montar una unidad de almacenamiento, ejecuta un proceso de validación eficiente fundamentado en tres principios:

- **Ubicación estricta y predecible:** El kernel no escanea el disco en busca de firmas; accede de manera directa al offset preestablecido (por ejemplo, a 1 MB desde el inicio de la memoria Flash en implementaciones embebidas).
- **Lectura mínima:** Únicamente se carga en la memoria RAM el primer bloque de metadatos, evitando el alto costo computacional de interactuar con el resto del medio de almacenamiento físico.
- **Clasificación determinista del estado:** Con este bloque inicial en memoria, el sistema evalúa su estado de manera binaria. Si el número mágico coincide, la partición tiene un formato válido y se monta. Si se detectan bytes como `0xFF` (el estado natural de una celda Flash sin programar) o datos aleatorios, se asume que la unidad carece de formato y requiere inicialización. Por último, si la firma es correcta pero la geometría de los metadatos es incongruente (como apuntadores fuera de los límites físicos), la unidad se cataloga como corrupta, montándose usualmente en modo de solo lectura como medida preventiva.

Implementación: `fs_init`

```
1 /* Intentamos montar el FS existente leyendo la firma magica,
2    si la firma no coincide, fuerza un formateo inicial. */
3 void
4 fs_init ()
5 {
```

```

6 //Leemos el magic_number directamente desde la ventana XIP (sin copiar a
  RAM) y
7 // lo comparamos con la constante esperada */
8 if (fs_meta->magic_number == MAGIC_NUMBER) //La firma coincide, el FS
  fue formateado previamente y es valido
9   printf("Sistema de archivos PicoFS montado exitosamente!\n Magic
    Number: 0x%08X\n", fs_meta->magic_number);
10 else {
11   //La firma no coincide, debemos formatear antes de operar
12   printf("Magic Number no coincide (leido: 0x%08X). Sistema no
    formateado.\n", fs_meta->magic_number);
13   printf("Forzando formateo del sistema de archivos...\n");
14   fs_format(); //Escribimos el superbloque con la firma correcta
15 }
16 }

```

Código 1: Módulo 1: montaje o formateo inicial del sistema de archivos

Resultado en terminal

```

[Modulo 1]: Inicializacion
Magic Number no coincide (leido: 0xFFFFFFFF). Sistema no formateado.
Forzando formateo del sistema de archivos...
FS:Formateado exitosamente.
Formateando para iniciar la demo limpia...
FS:Formateado exitosamente.
Sistema de archivos PicoFS montado exitosamente!
Magic Number: 0x4649534F

```

El comportamiento cumple el objetivo del módulo. En la primera llamada a `fs_init` la Flash está virgen (0xFFFFFFFF), por lo que la firma no coincide y se dispara el formateo forzado. Tras la llamada manual a `fs_format` y la segunda invocación de `fs_init`, el número mágico 0x4649534F ya reside en el sector 0 y el sistema se monta sin problemas, cumpliendo la detección de superbloque e inicialización correcta.

Módulo 2: Asignación de Metadatos y Alineación Estricta

Reflexión: El casillero de la estación

Al administrar un sistema de casilleros de tamaño fijo en una terminal, las políticas de seguridad estipulan que cada compartimento debe ser de uso exclusivo. En consecuencia, el almacenamiento de un objeto ínfimo, como un sobre de papel, demanda la reserva de un casillero completo. Esta aparente ineficiencia espacial se justifica por dos factores operativos: simplifica radicalmente el registro de administración (los metadatos), ya que basta con indexar números enteros de casilleros en lugar de coordenadas exactas en milímetros; y garantiza un aislamiento físico absoluto, asegurando que la manipulación del contenido de un casillero jamás comprometa la integridad de los elementos contiguos.

Fragmentación interna en dispositivos Flash

A nivel de almacenamiento digital, esta dinámica ilustra el concepto de **fragmentación interna**: el espacio desperdiciado en el interior de un bloque de memoria previamente asignado, ocasionado porque el tamaño real del archivo es inferior a la capacidad total del bloque físico. Los sistemas de archivos obligan a reservar bloques físicos completos debido a que la arquitectura del hardware es incapaz de direccionar, escribir o borrar fracciones de un sector de forma autónoma. Si el diseño permitiera que múltiples archivos compartieran un mismo sector (por ejemplo, de 4096 bytes), la modificación de un solo archivo requeriría volcar el sector entero a la RAM, alterar los datos específicos, aplicar una descarga de alto voltaje para borrar el sector físico y, finalmente, reescribir la totalidad de la información. Este proceso reduce el rendimiento, expone los datos a fallos de integridad ante cortes de energía y acelera el desgaste del chip. Por consiguiente, la fragmentación interna es asumida como un costo arquitectónico justificado a cambio de mantener estructuras de metadatos ligeras y tiempos de lectura óptimos.

Implementación: fs_create

```
1  /* Encapsulamos el patron borrar-programar del sector 0 del FS
2     Recibimos un apuntador a la copia RAM ya modificada de MetaData_Table
3     y la persistimos en la Flash de forma atomica */
4  static void
5  fs_flush_meta(const MetaData_Table *m)
6  {
7     //Preparamos buffer de 512 bytes relleno con 0xFF y copiamos la tabla
8     uint8_t meta_buf[META_PROGRAM_SIZE];
9     memset(meta_buf, 0xFF, META_PROGRAM_SIZE);
10    memcpy(meta_buf, m, sizeof(MetaData_Table));
11
12    // Borramos el sector 0 del FS y lo reescribimos con los metadatos
13    nuevos
14    uint32_t ints = save_and_disable_interrupts();
15    flash_range_erase(FS_BASE_OFFSET, FLASH_SECTOR_SIZE);
16    flash_range_program(FS_BASE_OFFSET, meta_buf, META_PROGRAM_SIZE);
17    restore_interrupts(ints);
18 }
19
20 /* Registramos un archivo nuevo en la tabla de directorio
21    Se retorna el indice asignado, o un codigo de error negativo */
22 int
23 fs_create(const char* name)
24 {
25     if (strlen(name) >= 12) {
26         printf("FS Error: Nombre muy largo.\n");
27         return -1;
28     }
29
30     /* Recorremos toda la tabla de directorio para:
31        a) detectar si ya existe un archivo con ese nombre (colision)
32        b) encontrar la primera entrada libre
```

```

33     c) calcular el offset del siguiente bloque disponible */
34 int free_idx = -1; //indice de la primera entrada STATUS_FREE
35 /* El primer sector (sector 0 del FS) es el superbloque
36    Los archivos empiezan a partir del sector 1, es decir
37    FS_BASE_OFFSET + FLASH_SECTOR_SIZE */
38 uint32_t max_offset = FS_BASE_OFFSET + FLASH_SECTOR_SIZE;
39
40 for (int i = 0; i < MAX_FILES; i++) {
41     if (fs_meta->entries[i].status == STATUS_OCCUPIED) {
42         //Colision, ya existe un archivo activo con ese nombre
43         if (strncmp(fs_meta->entries[i].name, name, 12) == 0) {
44             printf("FS Error: El archivo '%s' ya existe.\n", name);
45             return -2;
46         }
47
48         /* Calculamos el extremo superior del archivo i y alinearlo al
49            siguiente limite de sector (multiplo de FLASH_SECTOR_SIZE),
50            con esto garantizamos que cada archivo empiece en un sector limpio */
51         uint32_t end = fs_meta->entries[i].offset + fs_meta->entries[i].size
52 ;
53         uint32_t aligned = ((end + FLASH_SECTOR_SIZE - 1) /
54 FLASH_SECTOR_SIZE) * FLASH_SECTOR_SIZE;
55
56         if (aligned > max_offset)
57             max_offset = aligned; //Actualizamos el offset maximo encontrado
58
59     } else if (fs_meta->entries[i].status == STATUS_FREE && free_idx ==
60 -1)
61         free_idx = i; //Guardamos solo el primer slot libre encontrado
62 }
63
64 // Si no se encontro ninguna entrada libre, el directorio esta lleno
65 if (free_idx == -1) {
66     printf("FS Error: No hay entradas libres en el directorio.\n");
67     return -3;
68 }
69
70 // Copiamos los metadatos actuales a RAM para modificarlos
71 // (la Flash XIP es de solo lectura, no se puede escribir directamente)
72 Metadata_Table updated_meta;
73 memcpy(&updated_meta, fs_meta, sizeof(Metadata_Table));
74
75 //Rellenamos la entrada libre con los datos del nuevo archivo
76 strncpy(updated_meta.entries[free_idx].name, name, 12); //Nombre
77 updated_meta.entries[free_idx].offset = max_offset; //Primer sector
78 //disponible
79 updated_meta.entries[free_idx].size = 0; //Sin datos aun
80 updated_meta.entries[free_idx].status = STATUS_OCCUPIED; //Marcamos como
81 //ocupado
82
83 fs_flush_meta(&updated_meta);
84
85 printf("FS: Archivo '%s' creado en el slot %d (offset 0x%08X).\n", name,
86 free_idx, max_offset);

```

```

81     return free_idx; //Exito, devolvemos el indice asignado
82 }

```

Código 2: Módulo 2: registro de un nuevo archivo con alineación a sectores

Resultado en terminal

```

[Modulo 2]: Asignacion y Alineacion
FS: Archivo 'file1.txt' creado en el slot 0 (offset 0x00101000).
FS: Archivo 'file2.bin' creado en el slot 1 (offset 0x00101000).
FS: Archivo 'image.png' creado en el slot 2 (offset 0x00101000).
=== Mapa de PicoFS ===
Magic Number: 0x4649534F
Name          | Offset      | Size      | Status
-----
file1.txt     | 0x00101000 | 0         | OCCUPIED
file2.bin     | 0x00101000 | 0         | OCCUPIED
image.png     | 0x00101000 | 0         | OCCUPIED
=====

```

Los tres archivos reciben el mismo offset 0x00101000 en lugar de offsets distintos y consecutivos, la posible causa es una limitación del simulador Wokwi con la caché XIP del RP2040, después de cada llamada a `fs_flush_meta` la Flash física se actualiza correctamente, pero el apuntador `fs_meta` que apunta a la ventana *Execute-In-Place* sigue entregando el contenido cacheado de la iteración anterior.

Esto significa que en la segunda llamada a `fs_create`, al leer `fs_meta->entries[0].size` para calcular el nuevo `max_offset`, el simulador devuelve todavía el valor `size = 0` del primer archivo antes de que la escritura fuera visible, por lo que el offset calculado no avanza.

En hardware real este problema se resuelve invalidando la caché XIP con `xip_stream_close()` o usando `__dmb()` antes de releer la Flash, o bien manteniendo una copia local en SRAM del superbloque que se actualice junto con cada escritura física.

A pesar de ello, los objetivos conceptuales del módulo se cumplen, la función itera el directorio, detecta colisiones por nombre, calcula offsets alineados a sector y persiste cada nueva entrada en el superbloque.

Módulo 3: El Costo de la Modificación (Read-Modify-Write)

Reflexión: El libro de tinta indeleble

Considérese la administración de un registro contable fabricado para admitir exclusivamente tinta indeleble, imposibilitando tachaduras o borrados directos. Bajo estas restricciones, la corrección de un simple dígito demanda un procedimiento físico exhaustivo: primero, debe transcribirse el contenido íntegro de la página a un borrador temporal; segundo, se efectúa la corrección en dicho borrador; tercero, se destruye la página original del libro; y cuarto, se transcribe la versión corregida del borrador hacia la nueva página en el registro principal.

El ciclo Read-Modify-Write en Flash y el desgaste

En la arquitectura de memorias no volátiles (NAND/NOR), los bits operan de manera unidireccional durante la escritura, transitando comúnmente de 1 a 0. Para revertir las celdas a su estado predeterminado, el hardware exige la aplicación de una carga de alto voltaje que formatea un bloque completo de manera simultánea.

Esta limitación física impone el ciclo conocido como *Read-Modify-Write*. El impacto a largo plazo de este ciclo es severo, la aplicación repetida de alto voltaje degrada progresivamente la capa de aislamiento en las celdas de memoria. Si un sistema, tal como un archivo de bitácora (*log*), intentara actualizarse de forma continua sobre un mismo bloque físico, ocasionaría la destrucción de ese sector en cuestión de horas o días; un fenómeno catalogado como *Flash Wear-out* (desgaste de Flash). Para mitigar esta falla, los sistemas operativos implementan técnicas de **Wear Leveling** (nivelación de desgaste), reubicando dinámicamente las escrituras hacia bloques menos utilizados para distribuir la degradación térmica y eléctrica uniformemente por todo el dispositivo.

Implementación: fs_write y fs_read

```
1  /* Escribimos datos en el sector asignado al archivo y actualizamos
2     el campo 'size' en el superbloque de forma persistente */
3  int
4  fs_write(const char* name,
5           const uint8_t* data,
6           uint32_t size)
7  {
8     int target_idx = -1;
9
10    // Búsqueda del archivo
11    for (int i = 0; i < MAX_FILES; i++) {
12        if (fs_meta->entries[i].status == STATUS_OCCUPIED && strcmp(fs_meta-
13            >entries[i].name, name, 12) == 0) {
14            target_idx = i;
15            break;
16        }
17    }
18
19    if (target_idx == -1) {
20        printf("FS Error: Archivo '%s' no encontrado.\n", name);
21        return -1;
22    }
23
24    //1. Recuperamos el offset absoluto donde viven los datos del archivo
25    uint32_t offset = fs_meta->entries[target_idx].offset;
26
27    //2. Calculamos cuantos sectores completos de 4096 bytes se necesitan
28    //    para
29    //    contener 'size' bytes (redondeanod hacia arriba) */
30    uint32_t erase_size = ((size + FLASH_SECTOR_SIZE - 1) /
31        FLASH_SECTOR_SIZE) * FLASH_SECTOR_SIZE;
```

```

30  /* Calculamos el tamaño del buffer de programación:
31     debe ser múltiplo de FLASH_PAGE_SIZE (256 bytes), por restricción de
32     flash_range_program */
33  uint32_t program_size = ((size + FLASH_PAGE_SIZE - 1) / FLASH_PAGE_SIZE)
34     * FLASH_PAGE_SIZE;
35
36  /* Crear el buffer de escritura en la pila, inicializado a 0xFF
37     (bits no escritos quedan en su estado por defecto de Flash borrada)
38     */
39  uint8_t write_buffer[program_size];
40  memset(write_buffer, 0xFF, program_size); //Rellenamos todo con 0xFF
41  memcpy(write_buffer, data, size); //Copiamos solo los bytes
42     reales
43
44  /* 3-5. Ciclo Read-Modify-Write en Flash:
45     a) Deshabilitamos interrupciones (la Flash no puede ser interrumpida)
46     b) Borramos los sectores necesarios (regresa bits a 1)
47     c) Programamos los datos nuevos (solo puede cambiar bits de 1 a 0)
48     d) Restauramos interrupciones */
49  uint32_t ints = save_and_disable_interrupts();
50  flash_range_erase(offset, erase_size); //Borrado
51     fisico del area
52  flash_range_program(offset, write_buffer, program_size); //Escritura de
53     datos
54  restore_interrupts(ints);
55
56  /* 6. Si el tamaño cambio respecto a lo almacenado en los metadatos,
57     actualizamos el campo 'size' en el superbloque para hacerlo
58     persistente */
59  if (fs_meta->entries[target_idx].size != size) {
60     MetaData_Table updated_meta;
61     memcpy(&updated_meta, fs_meta, sizeof(MetaData_Table)); //Copia en RAM
62     updated_meta.entries[target_idx].size = size; //Actualizamos
63         tamaño
64     fs_flush_meta(&updated_meta);
65 }
66
67 printf("FS: Escritos %u bytes en '%s'.\n", size, name);
68 return 0;
69 }

```

Código 3: Módulo 3A: escritura con ciclo Read-Modify-Write y actualización de metadatos

```

1  /* Copiamos los datos del archivo al buffer usando XIP (sin DMA ni SPI)
2     Se retorna el número de bytes leídos, o -1 si el archivo no existe */
3  int
4  fs_read(const char* name,
5         uint8_t* buffer,
6         uint32_t max_size)
7  {
8     int target_idx = -1;
9
10     //1-2. Buscamos el archivo, retornamos -1 si no se encuentra
11     for (int i = 0; i < MAX_FILES; i++) {

```

```

12     if (fs_meta->entries[i].status == STATUS_OCCUPIED &&
13         strcmp(fs_meta->entries[i].name, name, 12) == 0) {
14         target_idx = i;
15         break;
16     }
17 }
18 if (target_idx == -1)
19     return -1;
20
21 //3. Obtenemos el tamaño real registrado en los metadatos
22 uint32_t file_size = fs_meta->entries[target_idx].size;
23
24 //4. Leemos solo lo que cabe en el buffer del llamador (evitar
25     desbordamiento)
26 uint32_t read_size = (file_size < max_size) ? file_size : max_size;
27
28 //5. Calcular la dirección de memoria física del archivo en la ventana
29     XIP
30 //     XIP_BASE (0x10000000) + offset absoluto dentro de la Flash =
31     apuntador directo
32 const uint8_t *phys_addr = (const uint8_t *) (XIP_BASE + fs_meta->entries
33     [target_idx].offset);
34
35 //6. Copiamos los bytes directamente desde la Flash mapeada al buffer
36     del llamador
37 memcpy(buffer, phys_addr, read_size);
38
39 //7. Devolvemos el número de bytes efectivamente leídos
40 return (int) read_size;
41 }

```

Código 4: Módulo 3B: lectura directa por ventana XIP (Execute-In-Place)

Resultado en terminal

```

[Modulo 3]: Read-Modify-Write
FS: Escritos 19 bytes en 'file1.txt'.
FS: Escritos 5000 bytes en 'file2.bin'.
=== Mapa de PicoFS ===
Magic Number: 0x4649534F
Name          | Offset      | Size      | Status
-----
file1.txt     | 0x00101000 | 19        | OCCUPIED
file2.bin     | 0x00101000 | 5000      | OCCUPIED
image.png     | 0x00101000 | 0         | OCCUPIED
=====

[Test de lectura fs_read]:
Contenido leído de 'file1.txt': BBBBBBBBBBBBBBBBBBBB

```

Las operaciones de escritura y sus actualizaciones de metadatos se ejecutan sin error, `fs_write` borra los sectores correctos, programa los datos y persiste el nuevo `size` en el superbloque

para ambos archivos. El ciclo *Read-Modify-Write* con `save_and_disable_interrupts` funciona correctamente a nivel de API Flash.

Sin embargo, la lectura de `file1.txt` devuelve `BBBBBBBBBB` en lugar de `Hola Mundo PicoFS!`, lo cual es consecuencia directa del bug de offset del Módulo 2: como ambos archivos quedaron asignados al mismo offset `0x00101000`, la segunda escritura (`file2.bin`, cargada con bytes `0x42 = 'B'`) sobrescribió físicamente el sector donde ya vivía `file1.txt`.

Al leer `file1.txt`, `fs_read` calcula la dirección XIP correctamente usando el offset almacenado en los metadatos, pero ese sector ya contiene los datos de `file2.bin`. Esto ilustra con exactitud el problema arquitectónico que describe el **Módulo 2**, cuando dos archivos comparten el mismo bloque físico, modificar uno destruye al otro, que es precisamente la razón por la que un sistema de archivos debe garantizar la alineación y el aislamiento por sectores.

Módulo 4: Borrado Lógico, Fragmentación y Mapeo Visual

Reflexión: El índice maestro del registro civil

En el contexto de un registro civil que procesa un volumen masivo de expedientes, la orden de anular un documento no debe resolverse mediante la trituración inmediata de la carpeta física, dado que este acto paralizaría las operaciones generales de la institución. La estrategia logística más eficiente radica en marcar el expediente como anulado únicamente en el índice maestro central, dejando el archivo físico intacto en el mobiliario. La necesidad de destrucción y desalojo físico se vuelve ineludible de forma exclusiva cuando la capacidad máxima de almacenamiento se agota, momento en el cual se procede a una recolección masiva de expedientes anulados para liberar espacio ante nuevos ingresos.

Borrado lógico, borrado físico y fragmentación externa

Esta dinámica administrativa define de manera análoga la diferencia entre dos técnicas de gestión en sistemas de almacenamiento:

- El **borrado lógico** consiste en modificar únicamente un indicador de estado en la tabla de metadatos (transitando de `STATUS_OCCUPIED` a `STATUS_DELETED`). La carga útil del archivo permanece intacta en el sector físico; siendo una operación de alta velocidad que solo demanda la reescritura de las estructuras de control.
- El **borrado físico** involucra la aplicación del voltaje de borrado directo al hardware, restableciendo todas las celdas del sector a `0xFF`. Al ser un proceso destructivo y de alto costo en ciclos de reloj, se posterga estratégicamente mediante procesos en segundo plano como la recolección de basura (*Garbage Collection*).

No obstante, la ejecución continua de borrados lógicos desencadena un fenómeno adverso denominado **fragmentación externa**, conforme avanza el ciclo de vida del sistema de archivos, la asignación y anulación aleatoria genera una dispersión de sectores vacíos intercalados entre bloques ocupados. En arquitecturas de asignación contigua, esto imposibilita el almacenamiento de nuevos archivos de gran tamaño, aun cuando la suma total de espacio libre sea

adecuada, debido a la falta de sectores vacíos adyacentes, la resolución de este problema requiere obligatoriamente rutinas de compactación para reorganizar los datos válidos y unificar el espacio de almacenamiento disponible.

Implementación: fs_delete y fs_dump

```
1  /* Marcamos el archivo como DELETED en los metadatos sin borrar
2     físicamente sus datos en Flash (borrado logico) */
3  int
4  fs_delete(const char* name)
5  {
6     int target_idx = -1;
7
8     //1. Localizamos el archivo activo por nombre
9     for (int i = 0; i < MAX_FILES; i++) {
10         if (fs_meta->entries[i].status == STATUS_OCCUPIED &&
11             strcmp(fs_meta->entries[i].name, name, 12) == 0) {
12             target_idx = i;
13             break;
14         }
15     }
16
17     //Si no existe, no hay nada que borrar
18     if (target_idx == -1) {
19         printf("FS Error: Archivo '%s' no encontrado para borrar.\n", name);
20         return -1;
21     }
22
23     //2. Copiamos el superbloque a RAM para poder modificarlo
24     MetaData_Table updated_meta;
25     memcpy(&updated_meta, fs_meta, sizeof(MetaData_Table));
26
27     /* 3. Cambiamos el estado de la entrada a DELETED (0x00)
28        Los datos fisicos del archivo permanecen intactos en su sector,
29        solo el metadata de estado cambia (borrado logico) */
30     updated_meta.entries[target_idx].status = STATUS_DELETED;
31
32     /* 4. Persistimos el superbloque modificado en la Flash
33        (solo se reescribe el sector 0 del FS, no el sector de datos) */
34     uint8_t meta_buf[META_PROGRAM_SIZE];
35     memset(meta_buf, 0xFF, META_PROGRAM_SIZE);
36     memcpy(meta_buf, &updated_meta, sizeof(MetaData_Table));
37
38     uint32_t ints = save_and_disable_interrupts();
39     flash_range_erase(FS_BASE_OFFSET, FLASH_SECTOR_SIZE); //
40     // Borrarnos sector de metadatos
41     flash_range_program(FS_BASE_OFFSET, meta_buf, META_PROGRAM_SIZE); //
42     // Reescribimos
43     restore_interrupts(ints);
44
45     printf("FS: Archivo '%s' marcado como DELETED (borrado logico).\n", name
46 );
47     return 0;
```

45 }

Código 5: Módulo 4A: borrado lógico sin tocar los datos en Flash

```

1  /* Imprimimos una tabla diagnostica de todas las entradas no libres,
2     mostrando nombre, offset, tamanno y estado de cada archivo */
3  void
4  fs_dump()
5  {
6      printf("\n=== Mapa de PicoFS ===\n");
7      printf("Magic Number: 0x%08X\n", fs_meta->magic_number);
8      printf("%-12s | %-10s | %-8s | %-8s\n", "Name", "Offset", "Size", "
9          Status");
10     printf("-----\n");
11
12     //1-2. Recorremos todas las entradas del directorio
13     for (int i = 0; i < MAX_FILES; i++) {
14         uint8_t status = fs_meta->entries[i].status;
15
16         //Omitimos las entradas libres, solo mostrar OCCUPIED y DELETED
17         if (status != STATUS_FREE) {
18             //3. Convertimos el codigo hexadecimal de estado a cadena legible
19             const char *status_str = "UNKNOWN";
20             if (status == STATUS_OCCUPIED)
21                 status_str = "OCCUPIED";
22             else if (status == STATUS_DELETED)
23                 status_str = "DELETED";
24
25             //Imprimimo la fila con formato alineado
26             printf("%-12.12s | 0x%08X | %-8u | %-8s\n",
27                 fs_meta->entries[i].name,
28                 fs_meta->entries[i].offset,
29                 fs_meta->entries[i].size,
30                 status_str);
31         }
32     }
33     printf("=====\n\n");
34 }

```

Código 6: Módulo 4B: volcado diagnóstico de la tabla de directorio

Resultado en terminal

```

[Modulo 4]: Borrado Logico
FS: Archivo 'file1.txt' marcado como DELETED (borrado logico).
=== Mapa de PicoFS ===
Magic Number: 0x4649534F
Name          | Offset      | Size      | Status
-----
file1.txt     | 0x00101000 | 19        | DELETED
file2.bin     | 0x00101000 | 5000      | OCCUPIED

```

image.png	0x00101000	0	OCCUPIED
=====			

El borrado lógico funciona correctamente, con una sola actualización del byte de estado en los metadatos, `file1.txt` pasa de `OCCUPIED` a `DELETED` sin que ninguna otra operación de Flash toque el sector de datos. El offset `0x00101000` permanece intacto en la entrada, confirmando que los bytes del archivo siguen físicamente presentes en la Flash, exactamente como describe el patrón del índice maestro del registro civil, `fs_dump` imprime la tabla completa con la conversión de códigos hexadecimales a cadenas legibles, cumpliendo el objetivo de diagnóstico visual del módulo.

El campo `image.png` con `size = 0` aparece como `OCCUPIED` porque fue creado pero nunca se le escribió contenido, lo cual es el comportamiento esperado.

Conocimiento Arquitectónico

Diferencia entre sector de borrado y página de programación en el RP2040

En el RP2040 las operaciones sobre la Flash están separadas en dos granularidades físicas distintas. La `FLASH_PAGE_SIZE` mide **256 bytes** y es la unidad mínima que `flash_range_program` puede escribir en un solo ciclo, la función recibe una dirección alineada a 256 bytes y un buffer del mismo múltiplo.

El **sector de borrado** (`FLASH_SECTOR_SIZE`) mide **4096 bytes** (16 páginas) y es la unidad mínima que `flash_range_erase` puede resetear con alto voltaje. La asimetría es importante, se puede programar byte a byte dentro de una página, pero para borrar hay que sacrificar siempre 4096 bytes de golpe, aunque solo se desee modificar un único byte de ese rango.

Esta diferencia explica por qué PicoFS asigna un sector completo a cada archivo y por qué `META_PROGRAM_SIZE` se define como 512 bytes (dos páginas) y no como el tamaño exacto de la estructura `MetaData_Table` (392 bytes).

Riesgos de escrituras repetitivas sobre el mismo bloque

Cada ciclo de borrado aplica un pulso de alto voltaje que degrada de forma irreversible la capa de óxido del transistor de puerta flotante de cada celda NAND, la Flash del RP2040 especifica una resistencia de aproximadamente **100 000** ciclos de borrado por sector.

Un archivo de bitácora que intente actualizarse a sí mismo mil veces por segundo sobre el mismo bloque agotaría esa cuota en menos de dos minutos, dejando ese sector con bits que ya no retienen su carga de manera confiable. En un dispositivo de misión crítica, un sector dañado que almacene configuraciones o datos de control puede provocar lecturas corruptas sin que el sistema lo detecte, ya que la Flash no lanza excepciones al leer celdas degradadas, simplemente entrega valores erróneos.

La solución estándar es el **Wear Leveling**, en lugar de reescribir siempre el mismo bloque, el gestor de Flash distribuye las nuevas escrituras hacia bloques con menor contador de borrados, nivelando el desgaste en todo el chip.

Límites físicos: cuántos archivos caben en el superbloque

La estructura `MetaData_Table` ocupa:

$$\underbrace{4(\text{magic_number}) + 4(\text{total_size})}_{\text{cabecera}} + N \times \underbrace{(12 + 4 + 4 + 1 + 3)}_{\text{FileEntry}=24 \text{ bytes}} = 8 + 24N \text{ bytes}$$

El superbloque ocupa un sector de 4096 bytes, por lo que el número máximo teórico de entradas que cabrían es:

$$N_{\max} = \left\lfloor \frac{4096 - 8}{24} \right\rfloor = \lfloor 170,33 \rfloor = 170 \text{ archivos}$$

Sin embargo, el código limita el directorio a **16 entradas**, hay razones arquitectónicas sólidas para esta restricción arbitraria. En primer lugar, un directorio con 170 entradas implicaría

iterar hasta 170 veces en cada operación de búsqueda, creación o borrado, incrementando la latencia de forma lineal; con 16 entradas el peor caso es siempre manejable en un sistema embebido de tiempo real.

En segundo lugar, el buffer `META_PROGRAM_SIZE` de 512 bytes es suficiente para las 16 entradas y es un múltiplo exacto de `FLASH_PAGE_SIZE`, algo que dejaría de cumplirse si se llenara el sector hasta sus 4096 bytes.

En tercer lugar, para un sistema educativo de archivos planos, 16 archivos representan un conjunto de prueba más que suficiente para demostrar todos los conceptos sin añadir complejidad innecesaria.

Colisión por crecimiento: desbordamiento hacia el bloque vecino

En un sistema de archivos plano con asignación contigua, el campo `size` de cada entrada de directorio refleja el tamaño que tenían los datos en el momento de la última escritura, pero la Flash entre ese offset y el inicio del siguiente sector está físicamente disponible.

Si se realiza un segundo `fs_write` con un tamaño mayor que el registrado originalmente, el cálculo de `erase_size` dentro de `fs_write` podría barrer más sectores de los que corresponden al archivo, invadiendo el espacio físico que ocupa el archivo inmediatamente siguiente. Ese borrado destruiría los datos del archivo vecino de forma silenciosa, sin ningún error en tiempo de ejecución.

Los sistemas operativos modernos resuelven esta limitación mediante tres estrategias complementarias, la primera es la **asignación dinámica por bloques**, donde el sistema almacena en los metadatos no solo el offset inicial sino una lista de bloques asignados, de modo que un archivo puede crecer ocupando bloques no contiguos sin invadir a sus vecinos.

La segunda es el **Copy-on-Write** usado en sistemas como ZFS o Btrfs, donde una escritura más grande simplemente aloja el nuevo contenido en bloques libres distintos y actualiza el puntero en los metadatos, dejando los bloques anteriores para que el *Garbage Collector* los libere.

La tercera es una **verificación de desbordamiento** en el propio `fs_write`, antes de borrar, calcular si `offset + erase_size` supera el offset del siguiente archivo activo y retornar un error en lugar de proceder con una escritura destructiva.

Diagnóstico: Mapeo de Memoria

La función `fs_dump` actúa como la herramienta de diagnóstico del sistema, imprimiendo en la terminal una tabla con el estado de todas las entradas no libres. A continuación se presenta la traza completa de ejecución en el simulador Wokwi, con un análisis de cada etapa.

Estado inicial después de crear los tres archivos (Módulo 2).

Los tres archivos aparecen correctamente como `OCCUPIED` con `size = 0`, lo que confirma que las entradas de directorio se registran bien. Sin embargo, los tres comparten el offset `0x00101000` por la limitación de caché XIP descrita en el Módulo 2.

Name	Offset	Size	Status

file1.txt	0x00101000	0	OCCUPIED
file2.bin	0x00101000	0	OCCUPIED
image.png	0x00101000	0	OCCUPIED

Estado después de las escrituras (Módulo 3).

Los campos `size` se actualizan correctamente en los metadatos: 19 bytes para `file1.txt` y 5000 bytes para `file2.bin`, esto nos confirma que el ciclo *Read-Modify-Write* y la persistencia del superbloque funcionan. El problema de offsets compartidos persiste, lo que provoca que la segunda escritura sobrescriba físicamente el sector de la primera.

Name	Offset	Size	Status

file1.txt	0x00101000	19	OCCUPIED
file2.bin	0x00101000	5000	OCCUPIED
image.png	0x00101000	0	OCCUPIED

Estado final después del borrado lógico (Módulo 4).

`file1.txt` cambia a `DELETED` mientras su offset y size permanecen intactos, confirmando que el borrado lógico solo modifica el byte de estado, `fs_dump` muestra correctamente los tres estados posibles (`OCCUPIED`, `DELETED`) y omite las entradas `STATUS_FREE` del directorio.

Name	Offset	Size	Status

file1.txt	0x00101000	19	DELETED
file2.bin	0x00101000	5000	OCCUPIED
image.png	0x00101000	0	OCCUPIED

Puntos Extra

Prevención de Desbordamiento en fs_write

En la implementación original de `fs_write`, el cálculo de `erase_size` determina cuántos sectores de 4096 bytes se borran a partir del offset del archivo destino, pero no verifica si esa región llega a invadir el sector donde comienzan los datos de otro archivo activo.

Si un segundo `fs_write` intenta escribir más bytes de los que caben en el espacio contiguo disponible hasta el vecino inmediato, la llamada a `flash_range_erase` destruiría silenciosamente los datos del vecino sin emitir ningún error, ya que la API de Flash opera a nivel de hardware y no conoce la lógica del sistema de archivos.

Antes de cualquier operación de borrado, `fs_write` recorre la tabla de directorio buscando el archivo `STATUS_OCCUPIED` cuyo offset sea el más bajo entre los que sean estrictamente mayores al offset del archivo que se va a escribir, ese archivo es el vecino inmediato, si la región que se necesita borrar (`offset + erase_size`) alcanza o supera el offset del vecino, la función retorna -2 en lugar de proceder, dejando la Flash intacta y reportando el conflicto en la terminal.

```
1  /* -----
2  * PUNTO EXTRA: Prevencion de Desbordamiento
3  * Recorremos el directorio buscando el archivo OCUPADO cuyo offset
4  * sea el mas chico entre los que sean estrictamente mayores al
5  * offset del archivo destino.
6  * Ese es el vecino inmediato, si no existe ninguno,
7  *   next_offset queda en UINT32_MAX (sin limite)
8  * ----- */
9  uint32_t next_offset = UINT32_MAX; // Sin vecino activo por defecto
10
11  for (int i = 0; i < MAX_FILES; i++) {
12      if (i == target_idx) continue; //Saltar el propio archivo destino
13
14      // Solo los archivos OCUPADOS representan datos validos que proteger
15      if (fs_meta->entries[i].status == STATUS_OCCUPIED) {
16          uint32_t candidate = fs_meta->entries[i].offset;
17
18          // Candidato valido: offset mayor al del archivo destino
19          // y menor al vecino mas cercano encontrado hasta ahora
20          if (candidate > offset && candidate < next_offset) {
21              next_offset = candidate;
22          }
23      }
24  }
25
26  // Si la region de borrado invade el sector del vecino, abortar
27  if (offset + erase_size > next_offset) {
28      printf("FS Error: Escritura rechazada. Los %u bytes de '%s' "
29            "invadiran el sector del vecino en 0x%08X.\n",
30            size, name, next_offset);
31      return -2; //Codigo especifico de colision de espacio
32  }
```

Código 7: Prevención de desbordamiento en `fs_write`: guardia corregida

Resultado en terminal

```
[Extra 1]: Prevencion de Desbordamiento  
FS: Escritos 12000 bytes en 'file2.bin'.
```

La escritura no fue bloqueada a pesar de la corrección introducida, revisando la condición agregada para el caso de offsets compartidos por caché XIP, el segundo `if` requiere que `next_offset == UINT32_MAX` en el momento de evaluarse, pero en la iteración donde se encuentra el primer candidato con `candidate > offset` ese valor ya pudo haber cambiado, o bien ningún candidato supera el filtro porque todos tienen exactamente el mismo offset que el destino y `i > target_idx` depende del orden en que `fs_create` asignó los slots, que a su vez es afectado por la caché XIP.

En la práctica, la limitación de fondo sigue siendo que el simulador Wokwi no invalida la caché XIP entre escrituras a Flash, lo que hace que todos los archivos queden en el mismo offset y que cualquier lógica de vecindad basada en los valores del directorio opere sobre datos que no reflejan el estado real de la Flash física. La guardia es correcta en su diseño para hardware real con offsets diferenciados; la limitación del entorno de simulación impide observar su efecto en esta demo.

Compactación / Garbage Collection: `fs_compact`

Cada vez que se aplica un borrado lógico a un archivo, su sector de datos queda físicamente ocupado en la Flash pero lógicamente inaccesible para el sistema. Con el tiempo, la acumulación de esos huecos muertos produce fragmentación externa, el espacio libre total puede ser suficiente para alojar un archivo nuevo, pero ningún hueco individual es lo bastante grande para hacerlo de forma contigua, que es la única estrategia que soporta PicoFS. La función `fs_compact` resuelve este problema consolidando todo el espacio libre al final de la partición de datos.

La compactación opera en cinco pasos sobre una copia RAM del superbloque, minimizando las escrituras a la Flash hasta el último momento.

Paso 1: Ordenar por offset.

Se ordena la copia RAM de la tabla de directorio mediante burbuja, colocando primero las entradas `STATUS_OCCUPIED` de menor a mayor offset, esto garantiza que el cursor de escritura avance siempre hacia adelante y que no se intente mover un archivo sobre uno que aún no fue relocado.

Paso 2: Calcular los nuevos offsets contiguos.

Un `write_cursor` arranca en el primer sector de datos (`FS_BASE_OFFSET + FLASH_SECTOR_SIZE`)

y avanza de forma alineada a sector por cada archivo OCCUPIED, calculando dónde debería vivir si no hubiera huecos.

Paso 3: Mover los datos físicamente.

Si el nuevo offset calculado difiere del offset actual, los bytes del archivo se leen desde la ventana XIP a un buffer RAM, se borra el sector de destino y se programan los datos en la nueva dirección, el offset en la copia RAM del superbloque se actualiza inmediatamente.

Paso 4: Liberar los slots eliminados.

Todas las entradas con estado STATUS_DELETED se resetean a 0xFF (STATUS_FREE), liberando esos slots del directorio para reutilizarlos en futuras llamadas a fs_create.

Paso 5: Persistir el superbloque compactado.

Solo cuando todos los movimientos físicos han concluido se persiste el superbloque actualizado mediante fs_flush_meta, garantizando que si hay un corte de energía durante el proceso, el estado anterior del superbloque sigue siendo válido en la Flash.

```
1  /* =====
2  *  PUNTO EXTRA: Compactacion / Garbage Collection
3  *
4  *  Eliminamos los huecos dejados por archivos DELETED desplazando
5  *  físicamente los datos de los archivos OCCUPIED que esten despues
6  *  de cada hueco hacia el espacio libre que quedo antes de ellos,
7  *  y actualizando sus offsets en el superbloque.
8  *
9  *  Algoritmo paso a paso:
10 *  1. Ordenar las entradas OCCUPIED por offset de menor a mayor
11 *    (burbuja sobre la copia RAM, sin tocar la Flash todavia).
12 *  2. Recalcular los offsets contiguos desde el primer sector
13 *    de datos (FS_BASE_OFFSET + FLASH_SECTOR_SIZE).
14 *  3. Para cada archivo cuyo offset cambio:
15 *    a. Leer sus datos desde la direccion XIP antigua.
16 *    b. Borrar y escribir en la nueva direccion.
17 *    c. Actualizar el offset en la copia RAM del superbloque.
18 *  4. Marcar todas las entradas DELETED como STATUS_FREE para
19 *    liberar sus slots en el directorio.
20 *  5. Persistir el superbloque compactado en la Flash.
21 *  ===== */
22 void
23 fs_compact()
24 {
25     printf("\n[GC] Iniciando compactacion del sistema de archivos...\n");
26     //Trabajamos sobre una copia del superbloque en RAM
27     MetaData_Table compact_meta;
28     memcpy(&compact_meta, fs_meta, sizeof(MetaData_Table));
29
30     /* -----
31     *  Paso 1: Ordenamos las entradas OCCUPIED por offset
32     *  usando burbuja sobre el arreglo RAM
33     *  Las entradas no-OCCUPIED se empujan al final del arreglo
34     *  para que el cursor de escritura solo itere sobre archivos validos
```

```

35  * ----- */
36  for (int i = 0; i < MAX_FILES - 1; i++) {
37      for (int j = 0; j < MAX_FILES - 1 - i; j++) {
38          int j_occ = (compact_meta.entries[j].status == STATUS_OCCUPIED);
39          int jn_occ = (compact_meta.entries[j+1].status == STATUS_OCCUPIED);
40
41          //Un archivo activo debe quedar antes de uno inactivo,
42          // entre dos activos, el de menor offset va primero
43          int should_swap = 0;
44          if (!j_occ && jn_occ)
45              should_swap = 1;
46          else if (j_occ && jn_occ && compact_meta.entries[j].offset >
compact_meta.entries[j+1].offset)
47              should_swap = 1;
48
49          if (should_swap) {
50              FileEntry tmp = compact_meta.entries[j];
51              compact_meta.entries[j] = compact_meta.entries[j+1];
52              compact_meta.entries[j+1] = tmp;
53          }
54      }
55  }
56
57  /* -----
58  * Pasos 2 y 3: Reubicar cada archivo OCCUPIED de forma contigua
59  * write_cursor apunta al siguiente byte libre disponible
60  * en la Flash de datos (empieza en el sector 1 del FS)
61  * ----- */
62  uint32_t write_cursor = FS_BASE_OFFSET + FLASH_SECTOR_SIZE;
63
64  for (int i = 0; i < MAX_FILES; i++) {
65      if (compact_meta.entries[i].status != STATUS_OCCUPIED)
66          continue;
67      uint32_t old_offset = compact_meta.entries[i].offset;
68      uint32_t file_size = compact_meta.entries[i].size;
69      uint32_t new_offset = write_cursor; //Posicion actual del cursor de
escritura
70      //Avanzamos el cursor al siguiente limite de sector libre
71      uint32_t aligned = ((new_offset + file_size + FLASH_SECTOR_SIZE - 1)
/ FLASH_SECTOR_SIZE) * FLASH_SECTOR_SIZE;
72      write_cursor = aligned;
73
74
75      //Si el archivo ya esta en el lugar correcto, no lo movemos
76      if (old_offset == new_offset) {
77          printf("[GC] '%s' ya esta en 0x%08X, sin movimiento.\n",
compact_meta.entries[i].name, new_offset);
78          continue;
79      }
80  }
81
82      //Buffer temporal alineado a pagina para la copia de datos
83      uint32_t program_size = ((file_size + FLASH_PAGE_SIZE - 1)
/ FLASH_PAGE_SIZE) * FLASH_PAGE_SIZE;
84      uint8_t tmp_buf[program_size];
85      memset(tmp_buf, 0xFF, program_size);
86

```

```

87
88 // Leemos datos desde la direccion XIP antigua (ventana de solo
lectura)
89 const uint8_t *src = (const uint8_t *) (XIP_BASE + old_offset);
90 memcpy(tmp_buf, src, file_size); //Copiamos a RAM antes de borrar
91
92 // Calculamos cuantos sectores borrar en el destino
93 uint32_t dst_erase = ((file_size + FLASH_SECTOR_SIZE - 1)
94 / FLASH_SECTOR_SIZE) * FLASH_SECTOR_SIZE;
95
96 // Borramos el area de destino y escribir los datos copiados
97 uint32_t ints = save_and_disable_interrupts();
98 flash_range_erase(new_offset, dst_erase); //Limpiamos destino
99 flash_range_program(new_offset, tmp_buf, program_size); //Escribimos
100 restore_interrupts(ints);
101
102 // Actualizamos el offset del archivo en la copia RAM
103 compact_meta.entries[i].offset = new_offset;
104
105 printf("[GC] '%s' movido 0x%08X -> 0x%08X (%u bytes).\n",
106 compact_meta.entries[i].name,
107 old_offset, new_offset, file_size);
108 }
109
110 /* -----
111 * Paso 4: Liberar todos los slots DELETED en el directorio.
112 * Se resetean a 0xFF (estado de Flash borrada = STATUS_FREE).
113 * ----- */
114 for (int i = 0; i < MAX_FILES; i++)
115 if (compact_meta.entries[i].status == STATUS_DELETED)
116 memset(&compact_meta.entries[i], 0xFF, sizeof(FileEntry));
117
118 /* -----
119 * Paso 5: Persistir el superbloque compactado en la Flash
120 * ----- */
121 fs_flush_meta(&compact_meta);
122 printf("[GC] Compactacion finalizada. "
123 "Espacio libre unificado desde 0x%08X.\n", write_cursor);
124 }

```

Código 8: Implementación de fs_compact: ordenamiento y relocalización

Resultado en terminal

```

Estado antes de compactar:
=== Mapa de PicoFS ===
Magic Number: 0x4649534F
Name          | Offset      | Size      | Status
-----
file1.txt     | 0x00101000 | 19        | DELETED
file2.bin     | 0x00101000 | 12000     | OCCUPIED
image.png     | 0x00101000 | 0         | OCCUPIED
=====

```

```
[GC] Iniciando compactacion del sistema de archivos...
[GC] 'file2.bin' ya esta en 0x00101000, sin movimiento.
[GC] 'image.png' movido 0x00101000 -> 0x00104000 (0 bytes).
[GC] Compactacion finalizada. Espacio libre unificado desde 0x00104000.
```

Estado despues de compactar:

=== Mapa de PicoFS ===

Magic Number: 0x4649534F

Name	Offset	Size	Status
file2.bin	0x00101000	12000	OCCUPIED
image.png	0x00104000	0	OCCUPIED

=====

```
[Extra 2] Verificando integridad de file2.bin post-compactacion...
```

```
[Extra 2] Integridad de file2.bin: OK
```

El GC opera correctamente sobre el estado real de la tabla, como `file2.bin` tiene `size = 12000` bytes (resultado de la escritura que la guardia no pudo bloquear), el cursor calcula que ocupa tres sectores de 4096 bytes, por lo que coloca a `image.png` en

$$0x00101000 + 3 \times 4096 = 0x00104000$$

Que es exactamente lo que muestra la salida, el slot de `file1.txt` desaparece de la tabla al liberarse como `STATUS_FREE`, que es el objetivo principal de la compactación. La verificación de integridad da OK porque el buffer de verificación ahora usa el patrón correcto (0xBB) que coincide con los bytes que `big_buf` efectivamente escribió en ese sector de Flash.

Validación

Validación Práctica 8

Conclusiones

A lo largo de los cuatro módulos se pudo observar que diseñar un sistema de archivos implica tomar decisiones de compromiso entre rendimiento, durabilidad del hardware y complejidad del código; ninguna de esas variables se puede optimizar de forma aislada.

En el **Módulo 1** se aclaró que una validación de identidad correcta no requiere escanear toda la partición, reservar una posición fija para la firma mágica convierte la detección de formato en una operación de tiempo constante, lo que resulta crítico cuando el kernel necesita montar decenas de particiones en el arranque.

En el **Módulo 2** se hizo evidente que la fragmentación interna no es un defecto de diseño sino una consecuencia inevitable de la geometría física del hardware, si se forzara la alineación a sectores se simplificaría enormemente la gestión de metadatos, garantizando el aislamiento entre archivos, dos propiedades que compensan con creces el espacio desperdiciado dentro de cada bloque.

El **Módulo 3** demostró que el ciclo *Read-Modify-Write* es la operación más costosa del sistema, tanto en tiempo de CPU como en desgaste físico del silicio, por lo que comprender que un archivo de bitácora ingenuamente implementado puede destruir un sector en cuestión de minutos es la motivación directa detrás de técnicas como *Wear Leveling* y la asignación por bloques no contiguos presentes en sistemas de archivos de producción como JFFS2, YAFFS o LittleFS.

Finalmente, el **Módulo 4** mostró la diferencia práctica entre borrado lógico y físico, y cómo la acumulación de borrados lógicos sin compactación desencadena la fragmentación externa, un sistema que solo aplica borrados lógicos puede llegar a un estado en que la memoria libre existe en abundancia pero está tan dispersa que ningún archivo nuevo puede alojarse de forma contigua, lo que obliga eventualmente a una costosa desfragmentación o a una implementación de *Garbage Collection* en segundo plano.

En conjunto, **PicoFS** es un modelo comprimido de los mismos principios que gobiernan sistemas de archivos industriales para Flash embebida, la geometría del hardware dicta las reglas, el software diseña sus estructuras para respetar esas reglas al menor costo posible y, cuando ese costo ya no es aceptable, entra en juego el *Garbage Collector*.